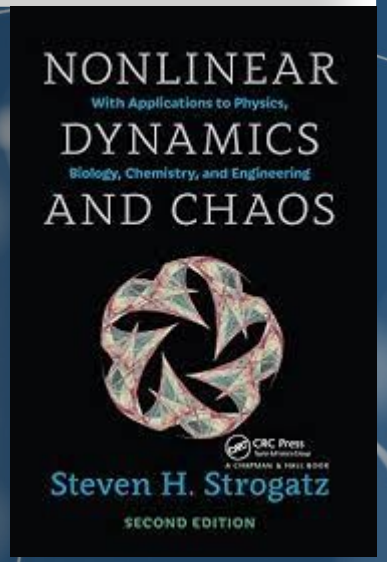
Module 3 (Solving Differential Equations: numerical techniques)



Example : Rabbit Vs Sheep, Lotka-Volterra, SIR dynamics

Jacobian analysis

Now for simpler example here we are showing a two coupled linear differential equation

$$\dot{Y} = Ay$$

$$\begin{bmatrix} y_1' \\ y_2' \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \end{bmatrix}$$

With Eigenvalues and Eigenvectors

λ_1 and v_1 λ_2 and v_2

The General Solution Takes The Form

$$\begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = C_1 e^{\lambda_1 t} v_1 + C_2 e^{\lambda_2 t} v_2$$

Jacobian analysis

- But what will happen if the system is nonlinear, and we want to understand whether the system shows stable equilibrium /fixed points.
- For example we have a non linear equation

$$\dot{X}_1 = f_1(x_1, x_2) \tag{3}$$

$$\dot{X}_2 = f_2(x_1, x_2) \tag{4}$$

- The fixed points or equilibrium points are (it is a point where the system converges at t tends to infinity) $= (x_1^*, x_2^*)$
- But how do we know that x_1^*, x_2^* are stable equilibrium points?

Jacobian analysis

- To calculate the stability we have to perturb the system around the equilibrium points. For a stable system a small perturbation will not disturb the fixed point from its initial position after the removal of perturbation
- To do so, we use Taylor series expansion around the equilibrium points
- If the small perturbation is δx_1,δx_2 then the Taylor series expansion of equation (3) and (4) is given below:-

$$\dot{\delta x_1} = f_1(x_1^* + \delta x_1, x_2^* + \delta x_2) = f_1(x_1^*, x_2^*) + \frac{\partial f_1}{\partial x_1} * \delta x_1 + \frac{\partial f_1}{\partial x_2} * \delta x_2 + \ldots \tag{5}$$

$$\dot{\delta x_2} = f_2(x_1^* + \delta x_1, x_2^* + \delta x_2) = f_2(x_1^*, x_2^*) + \frac{\partial f_2}{\partial x_1} * \delta x_1 + \frac{\partial f_2}{\partial x_2} * \delta x_2 + \ldots \tag{6}$$

Jacobian analysis

- From the last definition we can construct the Jacobian Matrix of nonlinear equation:

$$Jac = \begin{bmatrix} p_{11} & p_{12} \\ p_{21} & p_{22} \end{bmatrix} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} \end{bmatrix} \tag{7}$$

- This is linear dynamics that will give solution of decayed oscillation if the largest eigenvalue of the matrix Jac is negative

Rabbit Vs Sheep

Classic Lotka-Volterra model

Note :

- Each species would grow to its carrying capacity in absence of the other. (Recall Logistic growth equations : one self linear and the other negative nonlinear terms)
- Both eat grass.
- Rabbits have legendary ability to reproduce : thus higher intrinsic growth rate.
- Sheep usually beat rabbits when fighting for same tuft of grass.

Rabbit Vs Sheep

Classic Lotka-Volterra model

Note :

- Each species would grow to its carrying capacity in absence of the other.
  (Recall Logistic growth equations : one self linear and the negative nonlinear terms.)
- Both eat grass.
- Rabbits have legendary ability to reproduce : thus higher intrinsic growth rate
- Shee

When rabbits and sheep encounter each other, trouble starts. Sometimes the rabbit gets to eat, but more usually the sheep nudges the rabbit aside and starts nibbling (on the grass, that is). We'll assume that these conflicts occur at a rate proportional to the size of each population. (If there were twice as many sheep, the odds of a rabbit encountering a sheep would be twice as great.) Furthermore, we assume that the conflicts reduce the growth rate for each species, but the effect is more severe for the rabbits.

Rabbit Vs Sheep

Classic Lotka-Volterra model

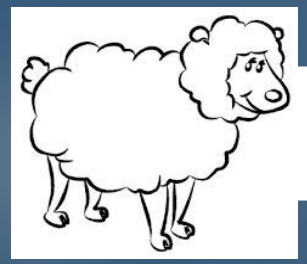
$$\dot{x} = x(3 - x - 2y)$$
$$\dot{y} = y(2 - x - y)$$

Rabbit Vs Sheep

Classic Lotka-Volterra model

Fast reproduction. Higher carrying capacity. Sheep threaten them.

$$\dot{x} = x(3 - x - 2y)$$
$$\dot{y} = y(2 - x - y)$$



Slower reproduction. Lower carrying capacity. Rabbits threaten less.

Rabbit Vs Sheep

Classic Lotka-Volterra model

$$\dot{x} = x(3 - x - 2y)$$
$$\dot{y} = y(2 - x - y)$$

Fixed Points: (0,0), (3,0), (0,2), (1,1)

Rabbit Vs Sheep

Classic Lotka-Volterra model

$$\dot{x} = x(3 - x - 2y)$$
$$\dot{y} = y(2 - x - y)$$

Fixed Points: (0,0), (3,0), (0,2), (1,1)

Jacobian

$$A = \begin{pmatrix} \frac{\partial \dot{x}}{\partial x} & \frac{\partial \dot{x}}{\partial y} \\ \frac{\partial \dot{y}}{\partial x} & \frac{\partial \dot{y}}{\partial y} \end{pmatrix} = \begin{pmatrix} 3 - 2x - 2y & -2x \\ -y & 2 - x - 2y \end{pmatrix}.$$

Rabbit Vs Sheep

Classic Lotka-Volterra model

$$\dot{x} = x(3 - x - 2y)$$
$$\dot{y} = y(2 - x - y)$$

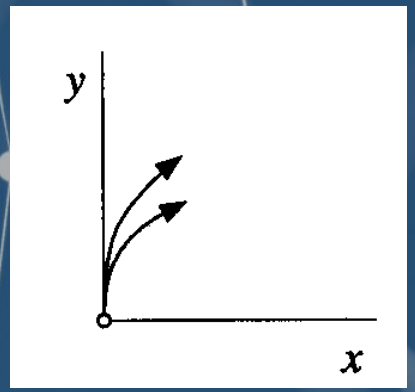
Fixed Points: (0,0), (3,0), (0,2), (1,1)

Jacobian

$$A = \begin{pmatrix} \frac{\partial \dot{x}}{\partial x} & \frac{\partial \dot{x}}{\partial y} \\ \frac{\partial \dot{y}}{\partial x} & \frac{\partial \dot{y}}{\partial y} \end{pmatrix} = \begin{pmatrix} 3 - 2x - 2y & -2x \\ -y & 2 - x - 2y \end{pmatrix}.$$

- **Case 1: $(0,0)$** $\lambda_1 = 3, \lambda_2 = 2$: ***Unstable node***
- **The trajectory will move away from origin parallel to the eigen vector $v_1 = (0,1)$ which is assigned for slow eigendirection ($\lambda_2 = 2$), and $v_2 = (1,0)$ for faster eigendirection ($\lambda_1 = 3$)**

This vector spans the y axis. The phase portrait near $(0,0)$ looks like



Rabbit Vs Sheep

$$\dot{x} = x(3 - x - 2y)$$
$$\dot{y} = y(2 - x - y)$$

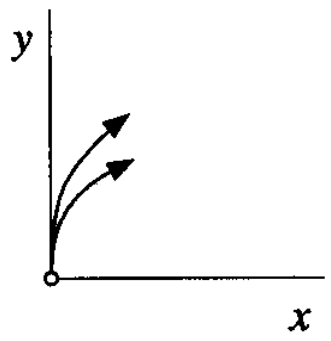
Fixed Points: (0,0), (3,0), (0,2), (1,1)

Jacobian

$$A = \begin{pmatrix} \frac{\partial \dot{x}}{\partial x} & \frac{\partial \dot{x}}{\partial y} \\ \frac{\partial \dot{y}}{\partial x} & \frac{\partial \dot{y}}{\partial y} \end{pmatrix} = \begin{pmatrix} 3 - 2x - 2y & -2x \\ -y & 2 - x - 2y \end{pmatrix}.$$

- **Case 1:** $(0,0)$ $\lambda_1 = 3, \lambda_2 = 2$: ***Unstable node***
- **The trajectory will move away from origin parallel to the eigen vector $v_1 = (0,1)$ which is assigned for slow eigendirection ($\lambda_2 = 2$), and $v_2 = (1,0)$ for faster eigendirection ($\lambda_1 = 3$,)**

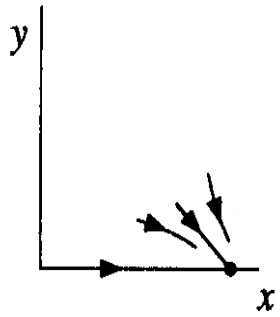
This vector spans the y axis. The phase portrait near $(0,0)$ looks like



$$\begin{pmatrix} -3 & -6 \\ 0 & -1 \end{pmatrix}$$

- **Case 2:** $(3,0)$ $\lambda_1 = -3, \lambda_2 = -1$: ***Stable node***

The trajectory will approach along the slow eigendirection ($\lambda_2 = -1$) spanned by $v = (3,-1)$. $\lambda_2 = -3$ :: $(1,0)$



$$A = \begin{pmatrix} \frac{\partial \dot{x}}{\partial x} & \frac{\partial \dot{x}}{\partial y} \\ \frac{\partial \dot{y}}{\partial x} & \frac{\partial \dot{y}}{\partial y} \end{pmatrix} = \begin{pmatrix} 3 - 2x - 2y & -2x \\ -y & 2 - x - 2y \end{pmatrix}.$$

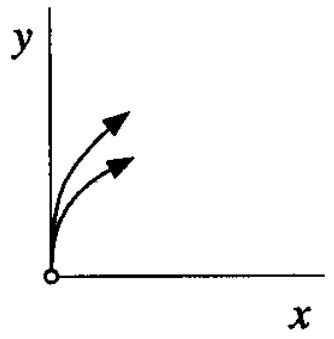
Rabbit Vs Sheep

$$\dot{x} = x(3 - x - 2y)$$
$$\dot{y} = y(2 - x - y)$$

Fixed Points: (0,0), (3,0), (0,2), (1,1)

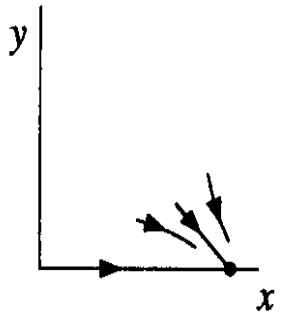
- **Case 1:** $(0,0)$ $\lambda_1 = 3, \lambda_2 = 2$: ***Unstable node***

  The trajectory will move away from origin parallel to the eigen vector $v = (0,1)$ which is assigned for slow eigendirection ($\lambda_2 = 0$). This vector spans the y axis. The phase portrait near $(0,0)$ looks like



- **Case 2:** $(3,0)$ $\lambda_1 = -3, \lambda_2 = -1$: ***Stable node***

  The trajectory will approach along the slow eigendirection ($\lambda_2 = -1$) spanned by $v = (3,-1)$; $\lambda_2 = -3$ :: $(1,0)$

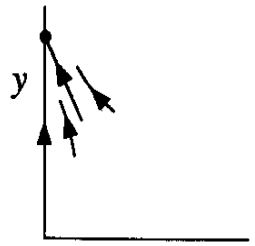


$$A = \begin{pmatrix} \frac{\partial \dot{x}}{\partial x} & \frac{\partial \dot{x}}{\partial y} \\ \frac{\partial \dot{y}}{\partial x} & \frac{\partial \dot{y}}{\partial y} \end{pmatrix} = \begin{pmatrix} 3 - 2x - 2y & -2x \\ -y & 2 - x - 2y \end{pmatrix}.$$

$$\begin{pmatrix} -1 & 0 \\ -2 & -2 \end{pmatrix}$$

- **Case 3:** $(0,2)$ $\lambda_1 = -1, \lambda_2 = -2$: ***Stable node***

  The trajectory will approach along the slow eigendirection ($\lambda_2 = -1$) spanned by $v = (1,-2)$; $\lambda_2 = -2$ :: $(0,1)$



Rabbit Vs Sheep

$$\dot{x} = x(3 - x - 2y)$$
$$\dot{y} = y(2 - x - y)$$

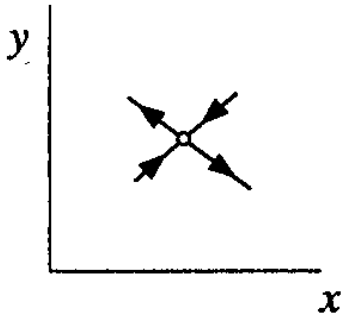
Fixed Points: (0,0), (3,0), (0,2), (1,1)

Jacobian

$$A = \begin{pmatrix} \frac{\partial \dot{x}}{\partial x} & \frac{\partial \dot{x}}{\partial y} \\ \frac{\partial \dot{y}}{\partial x} & \frac{\partial \dot{y}}{\partial y} \end{pmatrix} = \begin{pmatrix} 3 - 2x - 2y & -2x \\ -y & 2 - x - 2y \end{pmatrix}.$$

$(1,1)$: Then $A = \begin{pmatrix} -1 & -2 \\ -1 & -1 \end{pmatrix}$, which has $\tau = -2$, $\Delta = -1$, and $\lambda = -1 \pm \sqrt{2}$.

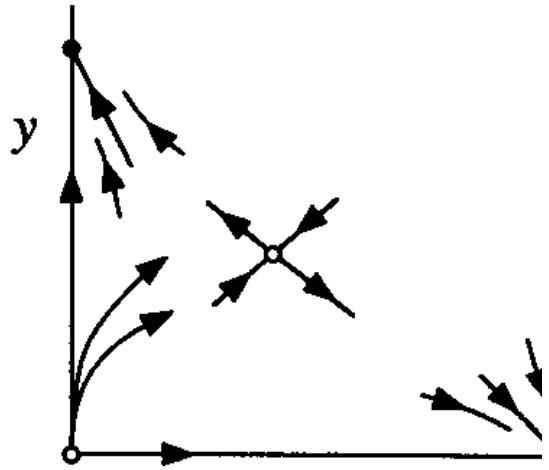
Hence this is a ***Saddle point.*** you can check, the phase portrait near $(1,1)$ is as shown in Figur

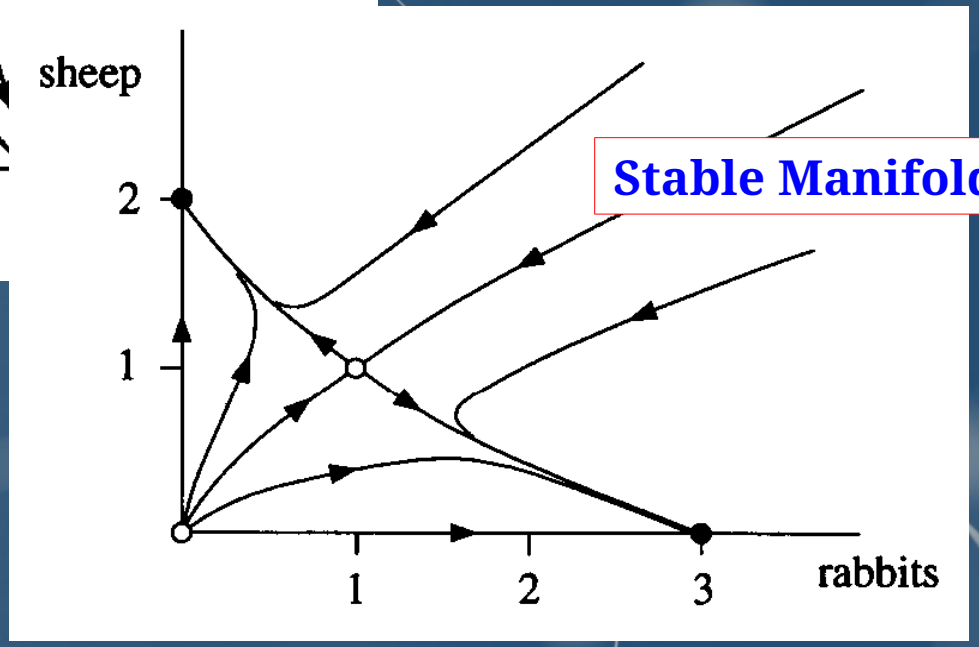


Rabbit Vs Sheep

$$\dot{x} = x(3 - x - 2y)$$
$$\dot{y} = y(2 - x - y)$$

Fixed Points: (0,0), (3,0), (0,2), (1,1)



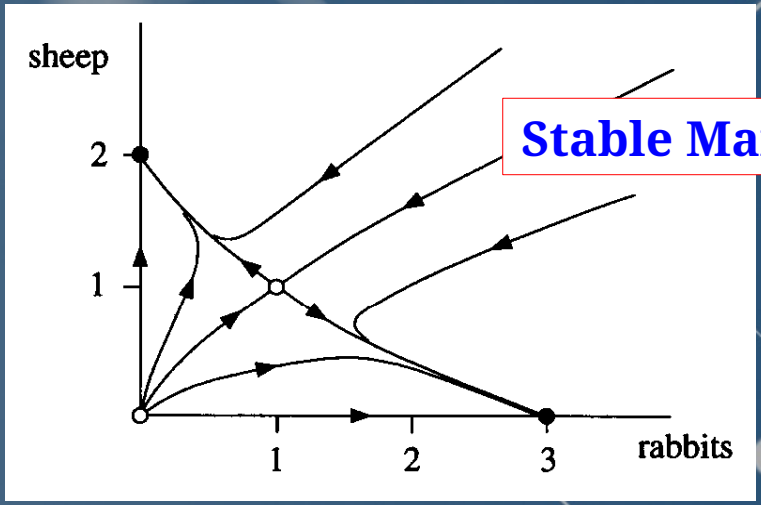


$$A = \begin{pmatrix} \frac{\partial \dot{x}}{\partial x} & \frac{\partial \dot{x}}{\partial y} \\ \frac{\partial \dot{y}}{\partial x} & \frac{\partial \dot{y}}{\partial y} \end{pmatrix} = \begin{pmatrix} 3 - 2x - 2y & -2x \\ -y & 2 - x - 2y \end{pmatrix}.$$

Rabbit Vs Sheep

$$\dot{x} = x(3 - x - 2y)$$
$$\dot{y} = y(2 - x - y)$$

Fixed Points: (0,0), (3,0), (0,2), (1,1)



- One species generally drives other species to extinction
- Below stable manifold, the trajectories lead to the sole existence of all rabbits
- Above the stable manifold, the trajectories lead to the extinction of rabbits.

Principle of competitive exclusion: Two species competing for same limited resource cannot coexist!!!!

Rabbit Vs Sheep: general model

We introduce **logistic growth for rabbits** and **self-limiting growth for sheep**:

$$\frac{dR}{dt} = aR(1 - \frac{R}{K}) - bRS$$

$$\frac{dS}{dt} = gS(1 - \frac{S}{M}) - cS$$

where:

- **Rabbits** R:
  - $aR(1 - R/K)$ → Logistic growth with **carrying capacity** K.
  - $-bRS$ → Rabbits are eaten by sheep.
- **Sheep** S:
  - $gS(1 - S/M)$ → Sheep follow **logistic growth** with **carrying capacity** M.
  - $-cS$ → Sheep have a natural death rate.

Here, g represents sheep reproduction based on internal factors (like birth rate and food availability), not dependent on rabbits.

Rabbit Vs Sheep: general model

Euler's Method Implementation

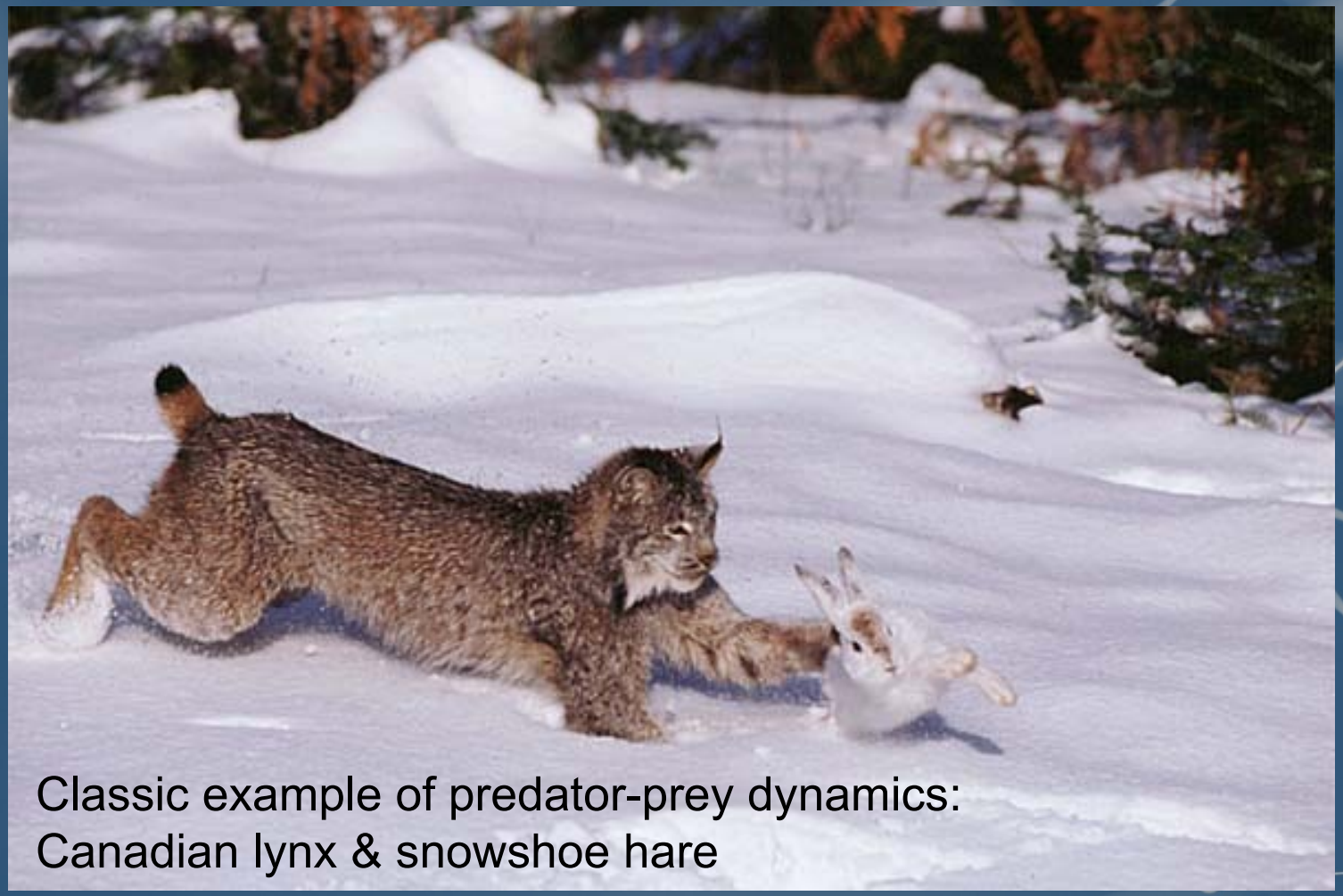
Using Euler's method, we update populations as:

$$R^{n+1} = R^n + \Delta t \cdot (aR^n(1 - \frac{R^n}{K}) - bR^n S^n)$$

$$S^{n+1} = S^n + \Delta t \cdot (gS^n(1 - \frac{S^n}{M}) - cS^n)$$

```
1. Initialize parameters:
   - Set a = [rabbit birth rate]
   - Set b = [predation rate]
   - Set c = [sheep death rate]
   - Set g = [sheep reproduction rate]
   - Set K = [rabbit carrying capacity]
   - Set M = [sheep carrying capacity]
   - Set Δt = [time step size]
   - Set t_final = [final simulation time]
   - Set R = [initial rabbit population]
   - Set S = [initial sheep population]

2. Compute the number of iterations:
   - N = t_final / Δt

3. Loop over each time step:
   for n = 0 to N-1:
     a. Compute rabbit population change:
        dR_dt = a * R * (1 - R / K) - b * R * S

     b. Compute sheep population change:
        dS_dt = g * S * (1 - S / M) - c * S

     c. Update populations:
        R = R + Δt * dR_dt
        S = S + Δt * dS_dt

     d. Ensure populations remain non-negative:
        if R < 0: R = 0
        if S < 0: S = 0

     e. Update time:
        t = (n + 1) * Δt

     f. (Optional) Store or print results:
        print("Time:", t, "Rabbits:", R, "Sheep:", S)
```

Predator-Prey model: Oscillatory dynamics



Classic example of predator-prey dynamics:
Canadian lynx & snowshoe hare

Coupled oscillation

Predator-Prey model: Oscillatory dynamics

- prey numbers increase, predator numbers increase…to a point where the predation causes population decline in the prey item..

Predator-Prey model: Oscillatory dynamics

In 1926 Volterra came up with a model to describe the evolution of predator and prey fish populations in the Adriatic Sea. Let $N(t)$ denote the prey population and $P(t)$ the predator population at time $t \geq 0$. He assumed that

Predator-Prey model: Oscillatory dynamics

In 1926 Volterra came up with a model to describe the evolution of predator and prey fish populations in the Adriatic Sea. Let $N(t)$ denote the prey population and $P(t)$ the predator population at time $t \geq 0$. He assumed that

1. In the absence of predators ($P = 0$) the per capita prey growth rate ($\frac{1}{N}\frac{dN}{dt}$) of the prey population N was constant, but fell linearly as a function of predator population P when predation was present ($P > 0$);

2. In the absence of prey ($N = 0$) the per capita growth rate of the predator ($\frac{1}{P}\frac{dP}{dt}$) was constant (and negative), and increased linearly with the prey population N when prey was present ($N > 0$).

Predator-Prey model: Oscillatory dynamics

In 1926 Volterra came up with a model to describe the evolution of predator and prey fish populations in the Adriatic Sea. Let $N(t)$ denote the prey population and $P(t)$ the predator population at time $t \geq 0$. He assumed that

1. In the absence of predators ($P = 0$) the per capita prey growth rate ($\frac{1}{N}\frac{dN}{dt}$) of the prey population N was constant, but fell linearly as a function of predator population P when predation was present ($P > 0$);

2. In the absence of prey ($N = 0$) the per capita growth rate of the predator ($\frac{1}{P}\frac{dP}{dt}$) was constant (and negative), and increased linearly with the prey population N when prey was present ($N > 0$).

$$\begin{aligned} \frac{1}{N}\frac{dN}{dt} &= a - bP \\ \frac{1}{P}\frac{dP}{dt} &= cN - d \end{aligned}$$

Predator-Prey model: Oscillatory dynamics

$$\frac{1}{N}\frac{dN}{dt} = a - bP$$

$$\frac{1}{P}\frac{dP}{dt} = cN - d$$

N	The population of prey at time t
P	The population of predator at time t
a	Rate constant on how fast prey reproduce themselves
b	Rate constant on how fast prey killed by predator
c	Rate constant on how fast predator reproduce when fed from prey
d	Rate constant on how fast predator die

Predator-Prey model: Oscillatory dynamics

$$\frac{1}{N}\frac{dN}{dt} = a - bP$$

$$\frac{1}{P}\frac{dP}{dt} = cN - d$$

Jacobian analysis

Jacobian matrix is formed by the partial derivatives of the ODE functions. In this way, we could approximately decouple the interfering parameters; such linearization allows us to study each parameter independently

$$J_F(x_1, \ldots, x_n) = \begin{bmatrix} \frac{\partial y_1}{\partial x_1} & \cdots & \frac{\partial y_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial y_m}{\partial x_1} & \cdots & \frac{\partial y_m}{\partial x_n} \end{bmatrix}$$

Predator-Prey model: Oscillatory dynamics

$$\frac{1}{N}\frac{dN}{dt} = a - bP$$

$$\frac{1}{P}\frac{dP}{dt} = cN - d$$

Jacobian analysis

Jacobian matrix is formed by the partial derivatives of the ODE functions. In this way, we could approximately decouple the interfering parameters; such linearization allows us to study each parameter independently

$$J_F(x_1, \ldots, x_n) = \begin{bmatrix} \frac{\partial y_1}{\partial x_1} & \cdots & \frac{\partial y_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial y_m}{\partial x_1} & \cdots & \frac{\partial y_m}{\partial x_n} \end{bmatrix}$$

$$\begin{pmatrix} a - bP & -bN \\ cP & cN - d \end{pmatrix}$$

Jacobian analysis

- The sign of the real parts of eigenvalues will determine the behaviours of surrounding points.
  - A negative value means they will be attracted towards the stationary point, because the change of x or y has an exponential decay.
  - A positive value means they will be repelled from the stationary point, since the change of x or y has an exponential increase.
  - A zero value, or eigenvalues appear as conjugate pairs, then they will oscillate around the stationary point, since there is not increase of decrease of x or y.
  - The magnitude of the eigenvalues will determine the strength of the change.

Jacobian analysis

- The sign of the real parts of eigenvalues will determine the behaviours of surrounding points.
  - A negative value means they will be attracted towards the stationary point, because the change of x or y has an exponential decay.
  - A positive value means they will be repelled from the stationary point, since the change of x or y has an exponential increase.
  - A zero value, or eigenvalues appear as conjugate pairs, then they will oscillate around the stationary point, since there is not increase of decrease of x or y.
  - The magnitude of the eigenvalues will determine the strength of the change.
- Hence, the stationary points can be characterized as follow:
  - If both the real parts of eigenvalues are negative, it is stable.
  - If one of the real parts of eigenvalues is positive, it is unstable.
  - If both the real parts of eigenvalues are zero, it is a center.

$$\frac{1}{N}\frac{dN}{dt} = a - bP$$

$$\frac{1}{P}\frac{dP}{dt} = cN - d$$

$$J_F(x_1, \ldots, x_n) = \begin{bmatrix} \frac{\partial y_1}{\partial x_1} & \cdots & \frac{\partial y_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial y_m}{\partial x_1} & \cdots & \frac{\partial y_m}{\partial x_n} \end{bmatrix}$$

$$\begin{pmatrix} a - bP & -bN \\ cP & cN - d \end{pmatrix}$$

Jacobian analysis and phase potrait

In Lotka-Volterra system, for the stationary point (0, 0) the Jacobian matrix is $\begin{bmatrix} a & 0 \\ 0 & -d \end{bmatrix}$, the eigenvalue are "a" & "-d", hence it is a saddle point.

For the stationary point (d/c, a/b), the Jacobian matrix is $\begin{bmatrix} 0 & -\frac{bd}{c} \\ \frac{ac}{b} & 0 \end{bmatrix}$, the eigenvalues are $i\sqrt{ca}$ & $-i\sqrt{ca}$, hence it is a center.

Jacobian analysis and phase potrait

$$\frac{1}{N}\frac{dN}{dt} = a - bP$$

$$\frac{1}{P}\frac{dP}{dt} = cN - d$$

$$J_F(x_1, \ldots, x_n) = \begin{bmatrix} \frac{\partial y_1}{\partial x_1} & \cdots & \frac{\partial y_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial y_m}{\partial x_1} & \cdots & \frac{\partial y_m}{\partial x_n} \end{bmatrix}$$

$$\begin{bmatrix} a - bP & -bN \\ cP & cN - d \end{bmatrix}$$

In Lotka-Volterra system, for the stationary point (0, 0) the Jacobian matrix is $\begin{bmatrix} a & 0 \\ 0 & -d \end{bmatrix}$, the eigenvalue are "a" & "-d", hence it is a saddle point.

For the stationary point (d/c, a/b), the Jacobian matrix is $\begin{bmatrix} 0 & -\frac{bd}{c} \\ \frac{ac}{b} & 0 \end{bmatrix}$, the eigenvalues are $i\sqrt{ca}$

```
function dydt = model(t,M,a,b,c,d)
n= M(1);
p = M(2);
dn_dt = n*(a-b*p); %%% prey
dp_dt = p*(c*n-d); %% predator


dydt = [dn_dt,dp_dt];
end
```

Jacobian analysis and phase potrait

$$\frac{1}{N}\frac{dN}{dt} = a - bP$$

$$\frac{1}{P}\frac{dP}{dt} = cN - d$$

$$J_F(x_1, \ldots, x_n) = \begin{bmatrix} \frac{\partial y_1}{\partial x_1} & \cdots & \frac{\partial y_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial y_m}{\partial x_1} & \cdots & \frac{\partial y_m}{\partial x_n} \end{bmatrix}$$

$$\begin{bmatrix} a - bP & -bN \\ cP & cN - d \end{bmatrix}$$

In Lotka-Volterra system, for the stationary point (0, 0) the Jacobian matrix is $\begin{bmatrix} a & 0 \\ 0 & -d \end{bmatrix}$, the

$\Big]$, the eigenvalues are $i\sqrt{ca}$

```
function dydt =
model(t,M,a,b,c,d)
n= M(1);
p = M(2);
dn_dt = n*(a-b*p); %%% prey
dp_dt = p*(c*n-d); %%
predator


dydt = [dn_dt,dp_dt];
```

```
a = 2/3;
b = 4/3;%0.1;
c =1;% 0.02;
d = 1;
hold on;
x0=1;y0=0.6;
init = [x0 y0]; %% 2/2 1.01/1.01,
t = linspace(0,100,30000)';
% Runge-Kutta 4th-Order Algorithm
y_euler = zeros(length(t), 2);
y_euler(1, :) = init;
h = t(2)-t(1); % Constant time step
for i = 2:length(t)
k1 =model(t(i-1), y_euler(i-1, :),a,b,c,d);
y_euler(i, :) = y_euler(i-1, :)+h*k1;
end
```

Jacobian analysis and phase potrait

$$\frac{1}{N}\frac{dN}{dt} = a - bP$$

$$\frac{1}{P}\frac{dP}{dt} = cN - d$$

$$J_F(x_1, \ldots, x_n) = \begin{bmatrix} \frac{\partial y_1}{\partial x_1} & \cdots & \frac{\partial y_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial y_m}{\partial x_1} & \cdots & \frac{\partial y_m}{\partial x_n} \end{bmatrix}$$

$$\begin{bmatrix} a - bP & -bN \\ cP & cN - d \end{bmatrix}$$

In Lotka-Volterra system, for the stationary point (0, 0) the Jacobian matrix is $\begin{bmatrix} a & 0 \\ 0 & -d \end{bmatrix}$, the

$\Bigg]$, the eigenvalues are $i\sqrt{ca}$

```
function dydt =
model(t,M,a,b,c,d)
n= M(1);
p = M(2);
dn_dt = n*(a-b*p); %%% prey
dp_dt = p*(c*n-d); %%
predator


dydt = [dn_dt,dp_dt];
```

```
a = 2/3;
b = 4/3;%0.1;
c =1;% 0.02;
d = 1;
hold on;
x0=1;y0=0.6;
init = [x0 y0]; %% 2/2 1.01/1.01,
t = linspace(0,100,30000)';
% Runge-Kutta 4th-Order Algorithm
y_euler = zeros(length(t), 2);
y_euler(1, :) = init;
h = t(2)-t(1); % Constant time step
for i = 2:length(t)
k1 =model(t(i-1), y_euler(i-1, :),a,b,c,d);
y_euler(i, :) = y_euler(i-1, :)+h*k1;
end
```

Predator-Prey model: Euler method

Two mass three spring systems: Euler method

Euler's Method Formulation

To use **Euler's method**, we convert the second-order ODEs into a system of first-order ODEs by defining velocities:

$$v_1 = \frac{dx_1}{dt}, \quad v_2 = \frac{dx_2}{dt}$$

$$\frac{dv_1}{dt} = \frac{-k_1 x_1 + k_2(x_2 - x_1)}{m_1}$$

$$\frac{dv_2}{dt} = \frac{-k_3 x_2 + k_2(x_1 - x_2)}{m_2}$$

Using **Euler's update rule**:

$$x_1^{n+1} = x_1^n + \Delta t \cdot v_1^n$$

$$x_2^{n+1} = x_2^n + \Delta t \cdot v_2^n$$

$$v_1^{n+1} = v_1^n + \Delta t \cdot \frac{-k_1 x_1^n + k_2(x_2^n - x_1^n)}{m_1}$$

$$v_2^{n+1} = v_2^n + \Delta t \cdot \frac{-k_3 x_2^n + k_2(x_1^n - x_2^n)}{m_2}$$

Two mass three spring systems: Euler method

```
1. Initialize parameters:
   - Set m1, m2 = [mass values]
   - Set k1, k2, k3 = [spring constants]
   - Set Δt = [time step size]
   - Set t_final = [final simulation time]
   - Set x1, x2 = [initial positions]
   - Set v1, v2 = [initial velocities]

2. Compute the number of iterations:
   - N = t_final / Δt

3. Loop over each time step:
   for n = 0 to N-1:
      a. Compute accelerations:
         a1 = (-k1 * x1 + k2 * (x2 - x1)) / m1
         a2 = (-k3 * x2 + k2 * (x1 - x2)) / m2

      b. Update velocities:
         v1 = v1 + Δt * a1
         v2 = v2 + Δt * a2

      c. Update positions:
         x1 = x1 + Δt * v1
         x2 = x2 + Δt * v2

      d. Update time:
         t = (n + 1) * Δt

      e. (Optional) Store or print results:
         print("Time:", t, "x1:", x1, "x2:", x2, "v1:", v1, "v2:", v2)
```